PYTHON PROGRAMMING
ASSIGNMENT 5 – CO5

DATA ACQUISITION, CLEANING, ANALYSIS AND AI APPLICATIONS

Submitted by

Dhaarani D

Registration Number

1913260041

Course: Python Programming (CSA08)

Course Outcome: CO5

Institution: SIMATS

Case Study: City Air-Quality Monitoring Network

GitHub Repository: CSA08-1913260041-Assignment5

GitHub Link: _____

Submission and Execution Note

This report provides the written design, pseudocode, completed TODO programs, implementation examples, and experimental scripts. Run the programs on your own computer, save your actual outputs and charts in the matching GitHub task folders, and replace any example benchmark values with measurements from your execution. Do not claim unexecuted timings as measured evidence.

Suggested repository structure

```
CSA08-1912260396-Assignment5/
  README.md
  requirements.txt
  Task01/
    cleaning_pipeline.py
    sample_air_quality.csv
    output_before_after.txt
    charts/
  Task02/
    sql_parameterisation.py
    adversarial_inputs/
  Task03/
    benchmark_methods.py
    profile_output.txt
  Task04/
    complaint_ai.py
    complaints.csv
    timing_index.txt
```

README details

• Name: K.S. Kanishka; Register number: 1912260396.

• Python version: record the version shown by python --version.

• List installed package versions using python -m pip freeze > requirements.txt.

• Include the run command, a short result for every task, output screenshots, and the working repository URL.

• Commit each task separately as it is completed; add faculty as collaborator if the repository is private.

PART A – DATA AND PROGRAM DESIGN

TASK A1 – DESIGN THE DATA ACQUISITION (7 Marks)

A1.1 – Inspect the Source

A station CSV is a comma-separated text file. Each record represents one station at one date and hour. The expected header is station,date,hour,pm25,pm10,no2,temperature,temperature_unit. A sample record is: Adyar,2026-10-01,08:00,48,72,18,30,C.

Expected data-quality issues:

Issue	Example	Why it is a problem
Missing value	pm25 is blank or NA	Cannot use as a numeric reading.
Negative reading	pm25 = -5	Physical reading is invalid.
Implausibly high value	pm25 = 1500 or temperature = 999	Likely sensor fault; define limits.
Duplicate record	Same station/date/hour appears twice	Can bias averages.
Wrong temperature unit	Temperature = 86 with unit Fahrenheit	Must convert to Celsius.
Malformed date/hour	2026/31/99 or 25:70	Cannot group chronologically.
Wrong type / extra spaces	' 48 ' or 'Adyar '	Needs trimming and conversion.
Wrong delimiter / encoding	Semicolon file or non-UTF-8 text	Parser may fail or read one column.
Missing column	No pm25 header	Required field unavailable.

Station spelling variation	'Adyar', 'ADYAR'	Creates false separate stations.

A1.2 – Design the Storage

Use two related SQLite tables. A station has one stable station identifier; readings reference it. The primary key on readings prevents duplicate station-time records.

```
CREATE TABLE stations (
    station_id INTEGER PRIMARY KEY,
    station_name TEXT NOT NULL UNIQUE,
    location TEXT
);

CREATE TABLE readings (
    station_id INTEGER NOT NULL,
    reading_date TEXT NOT NULL,       -- ISO YYYY-MM-DD
    reading_hour TEXT NOT NULL,       -- HH:MM, 24-hour
    pm25 REAL,
    pm10 REAL,
    no2 REAL,
    temperature_c REAL,
    PRIMARY KEY (station_id, reading_date, reading_hour),
    FOREIGN KEY (station_id) REFERENCES stations(station_id)
);
CREATE INDEX idx_readings_station_day
ON readings(station_id, reading_date);
```

Mapping: station text is trimmed and matched to stations.station_name; date and hour are parsed and formatted consistently; pollutant columns become REAL; Fahrenheit temperature is converted to Celsius before storage. The source unit should be retained in a staging/audit record if traceability is required.

A1.3 – CSV or SQL?

Board question	CSV + pandas each time	SQLite query
October mean PM2.5 at Adyar	Load all files, filter station/date, calculate mean.	SELECT AVG(pm25) with station and date predicates; index supports fil
Which station had the highest daily mean?	Load and group all relevant files in each run.	GROUP BY station_id, reading_date; aggregate in database.
Worst hours over a month	Load rows and group by hour in pandas.	SQL GROUP BY reading_hour with AVG/ORDER BY.

Alternative CSV-only design: use pandas.read_csv for each file, concatenate, clean, then filter/group for every question. A full scan is O(n) per question; indexed database lookup is commonly about O(log n) to locate rows plus the cost of returning/aggregating them (actual query plans vary). CSV uses disk persistently, while the DataFrame consumes RAM proportional to rows and columns. SQLite also uses disk, with indexes requiring extra storage.

TASK A2 – DESIGN CLEANING AND NORMALISATION (5 Marks)

A2.1 – Cleaning Rules

• Missing pollutant values: use a median for the same station and pollutant when enough valid samples exist; otherwise leave missing and exclude from that metric. Forward-fill is suitable only for short gaps and time-series continuity, not long outages.

• Impossible values: mark negative and out-of-range readings as invalid (e.g. PM2.5 < 0 or > 999 based on assignment rule); preserve the raw value in a quality log.

• Duplicates: retain one row for each station/date/hour; if duplicate values conflict, flag the record and apply a documented rule (e.g. median of valid duplicates).

• Fahrenheit: convert with C = (F – 32) × 5/9. Convert only stations documented as Fahrenheit.

• Invalid dates, hours, station names: parse strictly, normalize names and reject/log malformed rows.

A2.2 – Normalisation

Min–max scales a value to [0,1]: x' = (x – xmin)/(xmax – xmin). It is useful when features need a common bounded range, but is sensitive to outliers. If xmax = xmin, return 0 for all values or mark the feature constant.

Z-score standardisation: $z = (x - \mu)/\sigma$. It centers the data at zero and scales by standard deviation; useful for methods that benefit from standardized features. It is not bounded and can be affected by outliers. Calculate μ and σ from training/reference data only when building predictive models.

A2.3 – Formatting

• Date: YYYY-MM-DD; hour: HH:MM in 24-hour format; timezone documented consistently.

• Station names: trim whitespace, apply a canonical spelling/case, and map aliases to station IDs.

• Numbers: numeric float values, decimal point '.', no thousands separators; missing values represented as NULL/NaN rather than strings 'NA'.

Alternative dropping: remove rows with missing required values rather than filling. Daily averages may become less representative if missingness is not random (for example, a sensor fails during high pollution). Dropping and forward fill can be O(n) after scanning; median by sorting is O(n log n), while selection algorithms can compute a median in expected O(n). Space is O(n) if values are collected; streaming approaches can reduce it.

TASK A3 – DESIGN ANALYSIS AND VISUALISATION (5 Marks)

A3.1 – Choose the Charts

• Monthly PM2.5 trend: line chart with date on x-axis and mean PM2.5 on y-axis. Lines show temporal change and peaks.

• Station comparison: bar chart of station means (or box plot for distributions). Bars make differences between station averages easy to compare.

• Temperature versus PM2.5: scatter plot; each point is one paired observation. A trend line can indicate association, not causation.

• Worst hours: bar chart of hourly mean PM2.5 sorted by hour; a histogram is appropriate for distribution of individual readings, not ranking clock hours.

A3.2 – Daily Summary Table

Table daily_summary: station_id INTEGER, reading_date TEXT, pm25_min REAL, pm25_max REAL, pm25_mean REAL, valid_count INTEGER; primary key (station_id, reading_date). Produce it with SQL GROUP BY station_id, reading_date, using MIN, MAX, AVG and COUNT(pm25). Recompute after the cleaning/import job.

Alternative Python loops: scan n readings and update a dictionary keyed by (station, day) with min, max, sum and count. Time is O(n). Memory is O(g), where g is the number of station-day groups. SQL GROUP BY is also generally O(n) expected/linear scan plus grouping overhead; SQLite stores intermediate groups on disk or memory. Python and SQL both avoid storing a separate copy of all values when aggregating.

TASK A4 – DEBUGGING AND PROFILING (4 Marks)

A4.1 – Debugging Plan

• print(): inspect a small sample of raw rows, parsed values and cleaned rows during development.

• assert: check invariants such as pm25 >= 0, date format, no duplicate station-time key and temperature within the documented bounds.

• logging: use INFO for progress, WARNING for rejected data and ERROR for failures. Log counts and examples, not sensitive citizen content.

• pdb / IDE debugger: set a breakpoint inside the cleaning function; inspect the current row, parsed numeric value and branch condition.

A4.2 – Profiling Plan

• time.perf_counter(): measure wall-clock duration of complete pipeline stages.

• timeit: repeat small functions and report average/median duration, excluding one-time setup.

• cProfile: profile the full script and sort by cumulative time to locate the slowest functions.

Alternative print-only debugging: thousands of prints add I/O overhead and make logs noisy. Logging supports levels, formatting and handlers; set the level to WARNING/ERROR or disable the handler in production. Debugger breakpoints are removed/disabled after diagnosis. Avoid logging every record in the production path.

TASK A5 – DESIGN AI APPLICATION (4 Marks)

A5.1 – Sentiment and Summary

Lexicon scorer: tokenize lowercase text; positive words contribute +1, negative words –1, and neutral words 0. If the previous token is a negator (not, never, no), flip the current sentiment word's sign. Sum the scores: positive total = positive, negative total = negative, zero = neutral. This simple rule handles 'not good' but not all grammar or context.

Extractive summariser: split complaint text into sentences; remove common stop words; count important terms; score each sentence by the sum/frequency of its important words; select the top sentences and return them in original order. It reuses original sentences and does not generate new facts.

A5.2 – Basic Chatbot

Rule-based chatbot: detect intent using keywords such as 'PM2.5', 'today', and a station name. Parse the date (today in the configured timezone), map station to station_id, then execute a parameterized SQL query. If station/date is absent, ask a clarifying question. If no data exists, say so.

• Limitations: depends on exact keywords and known station names; struggles with misspellings, context, complex questions, and unseen intents; cannot explain causes or guarantee data freshness.

Alternative ML: Naive Bayes learns word probabilities from labeled complaints and may better classify varied phrasing if representative training data exists. A pretrained language model can handle context more flexibly but needs more memory and compute and can make unsupported claims. Lexicon scoring is lightweight (roughly O(words) per complaint, small constant memory); a trained model is also typically O(words × model features) at inference with model-dependent memory, and its accuracy must be evaluated on held-out local complaints.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND VALIDATE THE CLEANING PIPELINE (7 Marks)

B1.1 – Complete Code: TODO 1 and TODO 2

```
import csv
import io

RAW = """station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121
Guindy,2026-10-04,1500
"""

def load_clean(text):
    rows = []
    reader = csv.DictReader(io.StringIO(text))
    for row in reader:
        value = (row.get("pm25") or "").strip()
        if value == "" or value.upper() == "NA":
            continue
        try:
            pm = float(value)
        except ValueError:
            continue
        if pm < 0 or pm > 999:
            continue
        rows.append((row["station"].strip(), row["date"].strip(), pm))
    return rows

def min_max(values):
    values = list(values)
    if not values:
        return []
    low, high = min(values), max(values)
    if high == low:
        return [0.0 for _ in values]
    return [round((v-low)/(high-low), 3) for v in values]

clean = load_clean(RAW)
print(clean)
print(min_max([pm for _, _, pm in clean]))
```

Expected output (including the added >999 test row):

```
[('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)]
[0.0, 0.644, 1.0]
```

Before cleaning, the data includes NA, a blank, a negative value and a value above 999. After cleaning, only valid non-negative PM2.5 values in the allowed range remain. The upper-bound row (1500) is added to verify the new rule.

B1.2 – Compare with Design

The implementation follows the planned CSV parsing, numeric conversion and rejection rules. It adds a try/except around float conversion and handles a constant or empty list in min_max to avoid runtime errors. For a complete network pipeline, the same rules should be applied to PM10, NO2, date/hour and temperature.

B1.3 – Store and Query

```
import sqlite3

rows = [
    ("Adyar", "2026-10-01", 48.0),
    ("Guindy", "2026-10-01", 95.0),
    ("Guindy", "2026-10-03", 121.0),
]
con = sqlite3.connect("air_quality.db")
con.execute("CREATE TABLE IF NOT EXISTS aq (station TEXT, day TEXT, pm25 REAL, PRIMARY KEY(station, day))")
con.executemany("INSERT OR REPLACE INTO aq VALUES (?, ?, ?)", rows)
```

```
con.commit()

# 1. October average for Adyar
print(con.execute("SELECT AVG(pm25) FROM aq WHERE station=? AND day>=? AND day<?",
                  ("Adyar", "2026-10-01", "2026-11-01")).fetchone()[0])
# 2. Daily means by station
print(con.execute("SELECT station, day, AVG(pm25) FROM aq GROUP BY station, day").fetchall())
# 3. Station mean comparison
print(con.execute("SELECT station, AVG(pm25) FROM aq GROUP BY station ORDER BY AVG(pm25) DESC").fetchall())
con.close()
```

B1.4 – Visualise

For the full report, create the line, station-comparison bar, temperature-versus-PM2.5 scatter, and hourly bar charts with matplotlib. Each chart must include a title, labeled axes and units. Interpret the actual plotted results: identify dates with peaks, stations with higher means, whether temperature appears associated with PM2.5, and hours with the highest average. Avoid claiming causation from a scatter plot.

```
import matplotlib.pyplot as plt
# Example after loading a cleaned DataFrame named df:
daily = df.groupby("date")["pm25"].mean()
daily.plot(kind="line", marker="o")
plt.title("Daily Mean PM2.5")
plt.xlabel("Date"); plt.ylabel("PM2.5 (µg/m³)")
plt.tight_layout(); plt.savefig("daily_pm25.png"); plt.close()

station_mean = df.groupby("station")["pm25"].mean()
station_mean.plot(kind="bar")
plt.title("Mean PM2.5 by Station")
plt.xlabel("Station"); plt.ylabel("PM2.5 (µg/m³)")
plt.tight_layout(); plt.savefig("station_comparison.png"); plt.close()

df.plot(kind="scatter", x="temperature_c", y="pm25")
plt.title("Temperature vs PM2.5")
plt.xlabel("Temperature (°C)"); plt.ylabel("PM2.5 (µg/m³)")
plt.tight_layout(); plt.savefig("temperature_scatter.png"); plt.close()

hour_mean = df.groupby("hour")["pm25"].mean()
hour_mean.plot(kind="bar")
plt.title("Mean PM2.5 by Hour")
plt.xlabel("Hour"); plt.ylabel("PM2.5 (µg/m³)")
plt.tight_layout(); plt.savefig("hourly_mean.png"); plt.close()
```

Pandas alternative: read_csv, replace 'NA', to_numeric(errors='coerce'), filter values in [0,999], drop/fill missing data, then vectorize min-max. Both approaches scan O(n) rows. Pandas is usually faster for large data due to vectorized operations but uses more RAM (DataFrame plus temporary arrays); csv loops can be memory-light when streamed. Benchmark 1,000, 100,000 and 1,000,000 rows using time.perf_counter and record results from your machine.

TASK B2 – ENGINEER ADVERSARIAL DATA (5 Marks)

Complete Code

```
import sqlite3

readings = [
    ("Adyar", "2026-10-01", 48.0),
    ("Guindy", "2026-10-01", 95.0),
    ("Guindy", "2026-10-03", 121.0),
    ("Adyar", "2026-10-04", 60.0),
]

con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE aq (station TEXT, day TEXT, pm25 REAL)")
con.executemany("INSERT INTO aq VALUES (?, ?, ?)", readings)
con.commit()

def station_average(name):
    result = con.execute(
        "SELECT AVG(pm25) FROM aq WHERE station = ?", (name,)
    ).fetchone()[0]
    return result

print(station_average("Adyar"), station_average("Guindy"))
print(station_average("Adyar' OR '1'='1"))
con.close()
```

Expected output:

```
54.0 108.0
None
```

The quote-containing input is treated as a literal station name because it is bound as a value to the ? placeholder. No station has that exact name, so AVG returns SQL NULL, represented as Python None. The input cannot change the SQL structure.

Adversarial tests and interpretation

Input	Expected behavior / lesson
Missing column (e.g. no pm25)	Validate required headers; raise a clear error rather than silently returning empty output.
Wrong delimiter (semicolon)	Detect/configure delimiter; otherwise DictReader may see one combined header.
Different encoding	Try documented encodings or report UnicodeDecodeError; do not silently corrupt text.
Empty file	Check empty content/header and return a clear validation message.
Station name with quote	Parameterized query safely treats it as data.
Sarcasm: 'Great, another smoky day!'	Lexicon may call it positive because 'great' is not contextualized.
Double negative: 'not bad' / 'not not good'	One-word negation rule is simplistic and may not handle scope.
Mixed: 'clean road but smoke is bad'	Positive and negative terms may cancel to neutral despite complaint context.

Unsafe string concatenation example: if the SQL is constructed as f"... WHERE station = '{name}'", the supplied name can alter the WHERE clause and may match unintended rows. Parameterized SQL prevents this. executemany() reduces Python-to-SQL call overhead and sends a batch conveniently; both a loop of execute calls and executemany are O(n) overall, but executemany is typically faster. Exact timing depends on transaction boundaries and database.

TASK B3 – LOOPS VS LIBRARIES VS SQL (8 Marks)

B3.1 – Correctness on Small Data

Use identical cleaned data and group by (station, date). Compare the dictionaries/lists from each approach; assert equal keys and values within a floating-point tolerance.

```
from collections import defaultdict
import csv, sqlite3
import pandas as pd
from math import isclose
```

```
# Data columns: station,date,pm25
records = [
    ("Adyar", "2026-10-01", 48.0),
    ("Adyar", "2026-10-01", 52.0),
    ("Guindy", "2026-10-01", 95.0),
]

# 1) csv-style Python accumulation
acc = defaultdict(lambda: [0.0, 0])
for station, day, value in records:
    acc[(station, day)][0] += value
    acc[(station, day)][1] += 1
loop_result = {k: s/n for k, (s,n) in acc.items()}

# 2) pandas
df = pd.DataFrame(records, columns=["station","date","pm25"])
pandas_result = df.groupby(["station","date"])["pm25"].mean().to_dict()

# 3) SQL
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE aq(station TEXT, day TEXT, pm25 REAL)")
db.executemany("INSERT INTO aq VALUES(?,?,?)", records)
sql_result = {(s,d): m for s,d,m in db.execute(
    "SELECT station,day,AVG(pm25) FROM aq GROUP BY station,day")}
assert loop_result.keys() == pandas_result.keys() == sql_result.keys()
for key in loop_result:
    assert isclose(loop_result[key], pandas_result[key])
    assert isclose(loop_result[key], sql_result[key])
print(loop_result, pandas_result, sql_result)
db.close()
```

B3.2 – Generate and Measure

Generate approximately 8,760 records per station for each station count (1, 12, 120). Run each method several times, report median duration, and separate data generation from method execution. Use the same dataset for all three methods. The 120-station case contains about 1,051,200 rows.

```
import time, statistics
def benchmark(fn, repeats=3):
    durations = []
    for _ in range(repeats):
        start = time.perf_counter()
        fn()
        durations.append(time.perf_counter() - start)
    return statistics.median(durations)

# Call benchmark(loop_method), benchmark(pandas_method), benchmark(sql_method)
# on pre-generated data. Record the measured values in a table in your report.
```

Do not insert invented timings. Record your actual measurements in a table such as: station count | rows | Python-loop seconds | pandas seconds | SQL seconds.

B3.3 – Profile

```
import cProfile, pstats
cProfile.run("plain_python_method(records)", "profile.stats")
stats = pstats.Stats("profile.stats")
stats.strip_dirs().sort_stats("cumtime").print_stats(15)
```

The function with the largest cumulative time is the primary optimization candidate. The actual function depends on the implementation and dataset; include the cProfile output in the repository.

B3.4 – Engineering Trade-off

For quick one-time analysis, pandas groupby is concise and convenient for charting, provided memory is sufficient. For a daily automated report, store cleaned readings in SQLite and use indexed SQL aggregations, then export a small result to pandas/matplotlib. Plain Python is transparent and useful for streaming. Running sum/count aggregation is O(n) time and O(station × day groups) memory; retaining every reading first is O(n) memory. The running method is preferred when only means are needed.

TASK B4 – FINAL ENGINEERING DECISION (5 Marks)

B4.1 – Sentiment Scorer and Summariser

```
POSITIVE = {"good", "clean", "fast", "helpful", "improved"}
NEGATIVE = {"bad", "dirty", "slow", "rude", "smoke", "broken"}
NEGATORS = {"not", "never", "no"}

def score(text):
    words = text.lower().replace(".", " ").replace(",", " ").split()
    total = 0
    for i, word in enumerate(words):
        value = 1 if word in POSITIVE else -1 if word in NEGATIVE else 0
        if i > 0 and words[i-1] in NEGATORS:
            value *= -1
        total += value
    return total

def label(text):
    s = score(text)
    return "positive" if s > 0 else "negative" if s < 0 else "neutral"

complaints = [
    "Roads are clean and the bus was fast.",
    "Smoke from the factory, air is bad.",
    "The staff were not helpful.",
    "Not bad, service improved.",
]
for c in complaints:
    print(f"{label(c):8} | {c}")
```

Expected output:

```
positive | Roads are clean and the bus was fast.
negative | Smoke from the factory, air is bad.
negative | The staff were not helpful.
positive | Not bad, service improved.
```

A likely misclassification: 'The air is not bad, but smoke is still present.' The scorer may flip 'bad' to positive and then score 'smoke' as negative, resulting in neutral. The text's nuance and remaining concern are not understood.

Complaint set of 50 and agreement

Create at least 50 complaint sentences in complaints.csv. Manually label each complaint using your own reading, then compare with the program label. Agreement = (number of matching labels ÷ 50) × 100%. Report both the count and percentage. A meaningful example table should include complaint, human label, model label and match (yes/no). Do not claim a score until you have labeled and run the actual set.

Extractive summariser example

```
import re
from collections import Counter

STOP = {"the","a","an","and","is","are","to","of","in","for","on","was","were","it","this","that"}

def summarise(text, k=2):
    sentences = [s.strip() for s in re.split(r"(?<=[.!?])\s+", text) if s.strip()]
    words = [w.lower() for w in re.findall(r"\b[a-zA-Z]+\b", text)]
    freq = Counter(w for w in words if w not in STOP)
    scored = []
    for i, sentence in enumerate(sentences):
        terms = [w.lower() for w in re.findall(r"\b[a-zA-Z]+\b", sentence)]
        value = sum(freq[w] for w in terms if w not in STOP)
        scored.append((value, i, sentence))
    chosen = sorted(scored, key=lambda x: (-x[0], x[1]))[:k]
    return " ".join(sentence for _, _, sentence in sorted(chosen, key=lambda x: x[1]))
```

First-k sentence alternative: select the first k sentences without scoring. It is very fast, effectively O(k) selection once the text is available, but may miss the key information if it appears later. The frequency-based extractive method scans words and sentences, approximately O(words) time and O(vocabulary + sentences) space. Evaluate both on the same 50 complaints and compare whether the chosen sentences include the main issue, location and requested action.

B4.2 – Check the Database and Index

```
import time
```

```
# Measure the query before index, then create index and measure again.
query = "SELECT AVG(pm25) FROM aq WHERE station=? AND day>=? AND day<?"
args = ("Adyar", "2026-10-01", "2026-11-01")

def timed_query(con, repeats=100):
    start = time.perf_counter()
    for _ in range(repeats):
        con.execute(query, args).fetchone()
    return (time.perf_counter() - start) / repeats

before = timed_query(con)
con.execute("CREATE INDEX IF NOT EXISTS idx_aq_station_day ON aq(station, day)")
con.commit()
after = timed_query(con)
print("Before:", before, "After:", after)
```

The composite index can reduce the number of records examined for station/date filters, particularly as the table grows. For a tiny table, index creation and lookup overhead may make little visible difference. Record actual times and use EXPLAIN QUERY PLAN to confirm index use.

B4.3 – Production Decision

At the new scale, 100 additional stations reporting every 15 minutes generate 100 × 96 = 9,600 readings per day, before accounting for the original stations. Use a staged ingestion process: validate file/schema, parse and standardize units, reject or quarantine invalid rows with reason codes, deduplicate by station/time, and batch insert into SQLite or a server database. SQLite is suitable for a prototype or single-machine system; for concurrent writers, multiple services or larger operational demands, use PostgreSQL or another managed relational database.

• Store raw source files for audit; store normalized values and quality flags in the database.

• Create indexes on station/date and time; perform SQL daily/monthly aggregates; export aggregates for visualization.

• For 50,000 complaints/month, the lexicon is a useful baseline because it is cheap and transparent. Evaluate it against manually labeled data. If accuracy or sarcasm/context performance is inadequate, train/evaluate a Naive Bayes or other classifier using representative labeled complaints. A pretrained language model can be considered only with cost, latency, privacy and hallucination controls.

• Monitor ingestion failures, missingness, sensor drift, model agreement and query duration. Version the cleaning rules and preserve reproducible reports.

Final decision: use SQLite plus indexed SQL for the coursework-scale demonstration, pandas/matplotlib for analysis and plots, and a tested lexicon/extractive baseline for complaints. Move to a server database and a validated trained model only when measured scale, concurrency or accuracy requirements justify it.

Complexity Summary

Operation	Time	Space
CSV full scan per question	O(n)	O(n) if loaded to DataFrame; streaming can be O(1) extra
Indexed SQL filter	Approx. O(log n + r), where r is returned rows	Database index on disk; small query buffers
Cleaning scan	O(n)	O(n) if retaining cleaned rows; O(1) additional with streaming
Median by sorting	O(n log n)	O(n)
Daily group summary	O(n) expected	O(g) groups; g = station-days
Lexicon sentiment	O(w) per complaint	O(1) lexicon space for fixed vocabulary
Frequency extractive summary	O(w)	O(v + s), vocabulary v and sentences s
Python execute loop vs executemany	Both O(n), executemany usually lower overhead	O(batch) or driver-dependent

Conclusion

This assignment demonstrates how raw air-quality readings can be acquired, validated, cleaned, normalized, stored and queried. Parameterized SQL protects database queries, grouping and indexes support efficient summaries, and appropriate charts communicate trends and comparisons. Profiling and controlled experiments are necessary to

choose an implementation based on evidence. Simple sentiment and summarisation methods are explainable starting points, but their limitations must be measured before deployment.

References

• Python documentation: csv, sqlite3, statistics, re, time, timeit, cProfile and pdb modules.

• Pandas documentation: read_csv, DataFrame, groupby and data cleaning.

• Matplotlib documentation: plotting and chart labeling.

• SQLite documentation: SQL queries, indexes, parameter binding and query plans.